1 Повторные испытания в схеме Бернулли

Цель: научиться с помощью библиотеки `scipy.stats` рассчитывать вероятности в испытаниях Бернулли, а также применять приближенные формулы (Пуассона, Муавра-Лапласа) для случаев, когда точный расчет затруднителен.

Ввод [38]:
```
import numpy as np
from scipy.stats import binom, poisson, norm
import matplotlib.pyplot as plt
from math import sqrt

np.set_printoptions(formatter={'float_kind': '{:0.4f}'.format})
```

1.1 Схема Бернулли

> Последовательность n независимых испытаний, в каждом из которых событие A наступает с вероятностью p. Вероятность того, что событие наступит ровно m раз, задается формулой Бернулли: $P_n(m) = C_n^m \cdot p^m \cdot (1-p)^{n-m}$, где C_n^m – биномиальный коэффициент.

В `scipy` нет прямой функции для испытаний Бернулли, но для моделирования и анализа используются статистические распределения, такие как Биномиальное распределение (`scipy.stats.binom`), которое описывает вероятность m успехов в n испытаниях Бернулли с вероятностью успеха `p` .

Функции биномиального распределения:

- `binom.pmf(m, n, p)` – вероятность получить ровно m успехов в n испытаниях. *Пример*: `binom.pmf(2, 5, 0.5)` вычислит вероятность получить 2 успеха в 5 испытаниях, если вероятность успеха в каждом испытании равна 0.5.
- `binom.rvs(n, p, size=N)` – генерирует N случайных чисел, соответствующих биномиальному распределению (то есть, симулирует N серий из n испытаний).
- `binom.cdf(m, n, p)` – функция распределения, вероятность получить не более m успехов.

Замечание. `pmf` от *PMF* – Probability Mass Function.

Задача. Вероятность изготовления на станке стандартной детали равна $0,8$. Найти вероятности возможного появления бракованных деталей среди 5 отобранных.? Постройте график распределения вероятностей числа попаданий.

Ввод [26]:
```
# Параметры распределения
n = 5       # количество испытаний
p = 0.2     # вероятность брака в одном испытании
m = 3       # количество бракованных

prob = binom.pmf(m, n, p)
print(f"Вероятность ровно {m} бракованных из {n} отобранных: {prob:.4f}")
```

```
Вероятность ровно 3 бракованных из 5 отобранных: 0.0512
```

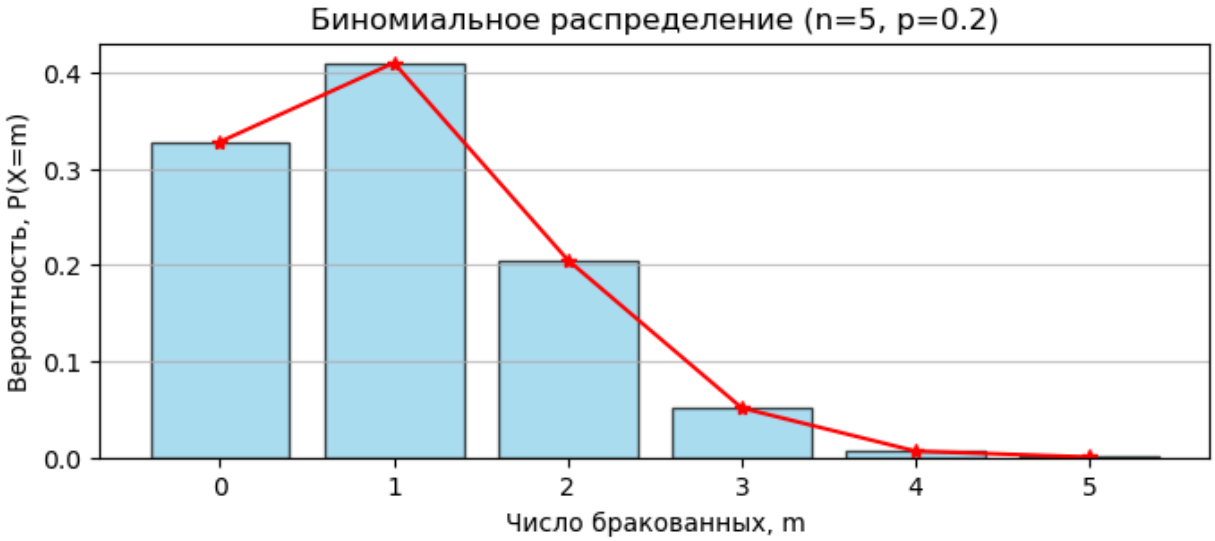
Ввод [29]:

```
# Создаем объект биномиального распределения
binomial_dist = binom(n, p)

# Создаем массив всех возможных значений m (от 0 до n)
m_values = np.arange(0, n + 1)

# Рассчитываем вероятности для каждого m
probabilities = binomial_dist.pmf(m_values)

# Строим столбчатую диаграмму
plt.figure(figsize=(8, 3))
plt.bar(m_values, probabilities, color='skyblue', edgecolor='black', alpha=0.7)
plt.plot(m_values, probabilities, 'r-*')
plt.title('Биномиальное распределение (n=5, p=0.2)')
plt.xlabel('Число бракованных, m')
plt.ylabel('Вероятность, P(X=m)')
plt.xticks(m_values)
plt.grid(axis='y', alpha=0.75)
plt.show()
```



Ввод [30]:

```
probabilities
```

Out[30]:

```
array([0.3277, 0.4096, 0.2048, 0.0512, 0.0064, 0.0003])
```

1.2 Распределение Пуассона

> Используется для аппроксимации биномиального распределения, когда n велико, а p мало (обычно $n > 50$, $p < 0.1$, и $\lambda = n \cdot p <= 10$). $P_n(m) \approx \frac{\lambda^m \cdot e^{-\lambda}}{m!}$, где $\lambda = n \cdot p$.

- `poisson.pmf(k, mu)` – значение функции Пуассона (вместо таблиц).

Задача. На факультете насчитывается 1825 студентов. Какова вероятность того, что 1 сентября является днем рождения одновременно 4 студентов?

Ввод [24]:
```
# Параметры для Пуассона
n = 1825
p = 1 / 365
lmb = n * p       # λ = n * p
m = 4

prob = poisson.pmf(k=m, mu=lmb)
print(f"Вероятность равна (Пуассон): {prob:.4f}")
```

```
Вероятность равна (Пуассон): 0.1755
```

Ввод [45]:
```
# Точное значение (биномиальное распределение)
binom.pmf(4, 1825, 1/365)
```

Out[45]:
```
np.float64(0.17561171906411857)
```

1.3 Теорема Муавра-Лапласа (локальная)

> **Локальная** используется для аппроксимации биномиального распределения, когда n велико, а p не слишком близко к 0 или 1 (обычно $n \cdot p \cdot (1-p) \geq 20$).
> $P_n(k) \approx \frac{\varphi(x)}{\sqrt{n \cdot p \cdot q}}$, где $x = \frac{m - n \cdot p}{\sqrt{n \cdot p \cdot q}}$, $q = 1 - p$, а $\varphi(x)$ – функция плотности стандартного нормального распределения.

- `norm.pdf(x, loc=0, scale=1)` – вычисляет значение функции плотности вероятности (PDF) для нормального (гауссовского) распределения в заданной точке x, используя параметры среднего значения `loc` и стандартного отклонения `scale`. Эта функция показывает вероятность того, что случайная величина примет конкретное значение в непрерывном распределении.

Чтобы найти PDF стандартного нормального распределения (со средним 0 и стандартным отклонением 1) в точке 0, вы можете использовать:

```
from scipy.stats import norm

print(norm.pdf(0))
```

Задача. Найти вероятность того, что событие A наступит ровно 80 раз в 400 испытаниях, если вероятность появления этого события в каждом испытании равна $0,2$.

Ввод [40]:
```
# Параметры для Муавра-Лапласа (локальной)
n = 400
p = 0.2
q = 1 - p
m = 80

x = (m - n * p) / sqrt(n * p * q)

prob = norm.pdf(x) / sqrt(n * p * q)
print(f"Вероятность равна (Мувр-Лаплас): {prob:.4f}")
```

```
Вероятность равна (Мувр-Лаплас): 0.0499
```

Ввод [43]:

```python
def local_moivre_laplace(m, n, p):
    """
    Вычисление вероятности P(X = m) с помощью локальной теоремы Муавра-Лапласа

    Parameters:
    m - число успехов
    n - количество испытаний
    p - вероятность успеха
    """
    q = 1 - p
    mu = n * p                      # математическое ожидание
    sigma = np.sqrt(n * p * q)      # стандартное отклонение
    x = (m - mu) / sigma            # стандартизированная величина
    phi_x = norm.pdf(x)             # плотность стандартного нормального распредел
    probability = phi_x / sigma     # пероятность по локальной теореме Муавра-Лапл

    return probability

local_moivre_laplace(80, 400, 0.2)
```

Out[43]: np.float64(0.04986778505017909)

Ввод [44]:

```python
# Точное значение (биномиальное распределение)
binom.pmf(80, 400, 0.2)
```

Out[44]: np.float64(0.04981327224985369)

1.4 Теорема Муавра-Лапласа (интегральная)

> **Интегральная** утверждает, что если в схеме Бернулли с n независимыми испытаниями вероятность успеха в каждом испытании равна p ($0 < p < 1$), то для любых действительных чисел a и b ($a < b$) при $n \to \infty$ справедливо:
> $P(a \le m \le b) \approx \Phi(b) - \Phi(a)$, где m – число успехов в n испытаниях, $q = 1 - p$,
> $\Phi(x)$ – функция стандартного нормального распределения.

- `norm.cdf(x, loc=0, scale=1)` – возвращает значение функции нормального распределения в точке x (*Cumulative Distribution Function*) для указанного среднего `loc` и стандартного отклонения `scale`.

Чтобы найти PDF стандартного нормального распределения (со средним 0 и стандартным отклонением 1) в точке 2.5, вы можете использовать:

```python
from scipy.stats import norm

print(norm.cdf(2.5))
```

Поправка на непрерывность

Поскольку биномиальное распределение дискретно, а нормальное – непрерывно, для улучшения точности используют поправку на непрерывность (*поправка Йейтса*):

Вероятность	Без поправки	С поправкой
$P(k = m)$	нет	$P(m - 0.5 \le X \le m + 0.5)$

Вероятность	Без поправки	С поправкой
$P(k \leq m)$	$P(X \leq m)$	$P(X \leq m + 0.5)$
$P(k \geq m)$	$P(X \geq m)$	$P(X \geq m - 0.5)$
$P(a \leq k$	$P(a \leq X$	$P(a - 0.5 \leq X \leq b$

Задача. Вероятность того, что деталь не прошла проверку ОТК равна $0,2$. Найти вероятность того, что среди 400 случайно отобранных деталей окажутся непроверенными от 70 до 100 деталей.

Ввод [52]:

```
n = 400
p = 0.2

# Границы интервала
a = 70
b = 100

mu = n * p
sigma = np.sqrt(n * p * (1-p))

x1 = (a - mu) / sigma
x2 = (b - mu) / sigma
prob = norm.cdf(x2) - norm.cdf(x1)
print(f"Вероятность: {prob:.4f}")
```

```
Вероятность: 0.8881
```

Ввод [49]:

```
# Параметры биномиального распределения
n = 400
p = 0.2
q = 1 - p

# Границы интервала
a = 70
b = 100

# Точный расчет через биномиальное распределение (может быть медленным для боль
# P(70 <= X <= 100) = CDF() - CDF()
binomial_dist = binom(n, p)
prob_binomial = binomial_dist.cdf(b) - binomial_dist.cdf(a - 1)
print(f"Точная вероятность (биномиальная): {prob_binomial:.4f}")

# Аппроксимация Муавра-Лапласа (через нормальное распределение)
# Находим матожидание и стандартное отклонение
mu = n * p
sigma = np.sqrt(n * p * q)

# Создаем объект нормального распределения
normal_dist = norm(loc=mu, scale=sigma)

# Вероятность в интервале [a, b] для НЕПРЕРЫВНОЙ величины.
# Расчет без поправки (менее точный)
prob_normal_approx = normal_dist.cdf(b) - normal_dist.cdf(a)
print(f"Вероятность (нормальная аппроксимация БЕЗ поправки): {prob_normal_appro

# Используем поправку на непрерывность (поправка Йейтса): для P(a<=X<=b) берем
prob_normal_approx = normal_dist.cdf(b + 0.5) - normal_dist.cdf(a - 0.5)
print(f"Вероятность (нормальная аппроксимация с поправкой): {prob_normal_approx
```

```
Точная вероятность (биномиальная): 0.9008
Вероятность (нормальная аппроксимация БЕЗ поправки): 0.8881
Вероятность (нормальная аппроксимация с поправкой): 0.9001
```

1.5 Задача 2.18.

В семье десять детей. Считая вероятности рождения мальчика и девочки равными между собой, определить вероятность того, что в данной семье:

- а) не менее трех мальчиков;
- б) не более трех мальчиков.

1.6 Задача 2.19.

Два равносильных противника играют в шахматы. Что более вероятно:

- а) выиграть 2 партии из 4 или 3 партии из 6;
- б) не менее 2 партий из 6 или не менее 3 партий из 6? (Ничьи в расчет не принимаются.)

1.7 Задача 2.20.

В банк отправлено 4000 пакетов денежных знаков. Вероятность того, что пакет содержит недостаточное или избыточное число денежных знаков, равна $0,0001$. Найти вероятность того, что при проверке будет обнаружено:

- а) три ошибочно укомплектованных пакета;
- б) не более трех пакетов.

1.8 Задача 2.21.

Строительная фирма, занимающаяся установкой летних коттеджей, раскладывает рекламные листки по почтовым ящикам. Прежний опыт работы компании показывает, что примерно в одном случае из двух тысяч следует заказ. Найти вероятность того, что при размещении 100 тыс. листков число заказов будет:

- а) равно 48;
- б) находиться в границах от 45 до 55.

1.9 Задача 2.26.

Вероятность того, что перфокарта набита оператором неверно, равна $0,1$. Найти вероятность того, что:

- а) из 200 перфокарт правильно набитых будет не меньше 180;
- б) у того же оператора из десяти перфокарт будет неверно набитых не более двух.

1.10 Задача 2.28.

При обследовании уставных фондов банков установлено, что пятая часть банков имеют уставный фонд свыше 100 млн руб. Найти вероятность того, что среди 1800 банков имеют уставный фонд свыше 100 млн руб.:

- а) не менее 300;
- б) от 300 до 400 включительно.

1.11 Задача 2.29.

Сколько нужно взять деталей, чтобы наивероятнейшее число годных деталей было равно 50, если вероятность того, что наудачу взятая деталь будет бракованной, равна $0,1$?

1.12 Задача 2.25.

Известно, что в среднем 60% всего числа изготовляемых заводом телефонных аппаратов является продукцией первого сорта. Чему равна вероятность того, что в изготовленной партии окажется:

- а) 6 аппаратов первого сорта, если партия содержит 10 аппаратов;

- б) 120 аппаратов первого сорта, если партия содержит 200 аппаратов?

2 Домашняя работа

Самостоятельно надо:

- разобрать решение задач §2.4 в учебнике Кремера;
- решить $2.22, 2.27, 2.23, 2.25, 2.30$;
- подготовиться к контрольной работе.

Ввод [ ]: